Zero-Shot Style Extraction and Closed-Form LoRA Distillation from Text-Conditioning Pathways

A CPU-Feasible Reformulation for Krea 2 RAW

A methodological redesign of velocity-field style extraction
for machines without a GPU

Abstract

Low-Rank Adaptation (LoRA) is the standard mechanism for attaching a reusable visual style to a text-to-image foundation model. Conventional style LoRAs are trained from captioned images and therefore inherit the cost of dataset curation and GPU-based optimization. A recent feasibility proposal suggested a different route: compare a model's own responses to a content prompt and a content-plus-style prompt, treat the difference as a style direction, and distill that direction into a LoRA without external images. That proposal operated on the flow-matching velocity field of Krea 2 RAW, a 12-billion-parameter diffusion transformer. The idea is conceptually sound, but the computational premise is not. A single forward pass of a 12B DiT is already impractical on CPU; back-propagating through the same model to fit LoRA weights is not a CPU research pipeline.

This paper redesigns the method around a cheaper and more local hypothesis. If a pretrained model already understands a named style, the style first appears as a shift in the text-conditioning pathway, and only afterwards as a change in the image velocity field. Krea 2 injects language through a small stack of linear maps — `txt_in` and `text_fusion` — before the packed text and image tokens enter the DiT blocks. Those maps can be read from disk without running the transformer, and a rank-1 or low-rank update that absorbs the text-feature shift can be written in closed form. The resulting adapter is a portable LoRA that can be validated on a hosted generator such as Tensor.Art or Krea Turbo. The method does not claim to invent unseen styles, nor to match the high-frequency fidelity of an image-trained LoRA. It asks a narrower question: whether an already-known style concept can be converted from an explicit prompt token into a default, adapter-controlled prior without a GPU and without a style image dataset.

1. Introduction

Modern text-to-image models store a large inventory of visual styles in their pretraining. A user who writes *a watercolor tiger walking through a jungle* is not teaching the model watercolor; the model already associates that word with a cluster of rendering behaviors. Conventional LoRA training ignores this fact. It collects images, captions them, and optimizes a low-rank update until the model reproduces the dataset. The pipeline works, but it is expensive, and it leaves open whether the adapter captured a general style concept or the statistics of a particular folder of pictures.

The original feasibility study that motivates this paper proposed to invert the supervision. Instead of images teaching the model, the model would teach an adapter. For matched latents and timesteps, one would compute the velocity difference

$$D_s = v_\theta(x_t, t, P_{c+s}) - v_\theta(x_t, t, P_c)$$

between a content-plus-style prompt and a content-only prompt.

If D_s contained a content-independent component, that component could be distilled into $\Delta W \approx BA$. The scientific question remains interesting. The engineering plan does not survive contact with a CPU-only machine. Krea 2 RAW is a dense 12B DiT. Collecting velocity tensors across concepts, seeds, and timesteps requires hundreds of full transformer forwards. Fitting LoRA against those tensors requires backward passes through the same frozen backbone. Avoiding VAE decoding, as the original study suggested, removes only a minor cost.

The present paper keeps the original research question and changes the locus of extraction. Style-conditioned behavior is still treated as a difference between paired prompts. The difference is measured in the text-conditioning pathway rather than in the image velocity field. The adapter is constructed by a closed-form low-rank update of the linear maps that send text features into the DiT, not by gradient descent on the DiT itself. Visual evaluation remains external.

1.1 Research question

Can a reusable Krea 2 style LoRA be obtained from a pretrained model by comparing text-encoder responses to content prompts and content-plus-style prompts, writing a closed-form low-rank update into the text-injection layers, and doing so without a GPU and without a style image dataset?

The question splits into five operational subquestions.

RQ1. Does the official text encoder of Krea 2 produce a stable, cross-content shift when a known style phrase is added to a prompt?

RQ2. Can that shift be separated from residual semantic effects of the extra tokens?

RQ3. Can the shift be absorbed into a low-rank update of `txt_in` and `text_fusion` without running the DiT?

RQ4. Does the resulting LoRA induce the target style on prompts that no longer contain the style word?

RQ5. Can the entire extraction and packaging pipeline run on CPU RAM, with image generation deferred to a hosted endpoint?

1.2 Contributions

This work makes three methodological contributions. First, it diagnoses why velocity-field self-distillation is not a CPU research method on a 12B flow-matching DiT, and it corrects an inconsistency in the original distillation objective. Second, it proposes a text-conditioning reformulation whose only local model is the official Qwen3-VL text encoder in text-only mode, plus a handful of linear weight matrices lazily read from the RAW checkpoint. Third, it specifies a closed-form rank-1 and thin-SVD construction that emits a platform-compatible LoRA, together with a four-condition external evaluation protocol that separates default-style behavior from explicit style prompting.

2. Background and Related Work

2.1 LoRA for text-to-image models

LoRA parameterizes a weight update as $\Delta W = BA$, with B $R^{d \times r}$ and A $R^{r \times k}$, $r << \min(d, k)$. For diffusion and flow-matching transformers, the usual targets are attention projections and often MLP and conditioning maps. Official Krea 2 guidance recommends training LoRAs on the undistilled RAW checkpoint and applying them at inference on the 8-step Turbo checkpoint. Default training recipes wrap all 264 Linear layers of the DiT at rank and alpha 32. That configuration is appropriate for image-supervised style and character adapters. It is the wrong default for a CPU extraction method, because it assumes the DiT will be present in the training loop.

2.2 Model-as-teacher style adapters

Concept Sliders demonstrated that a frozen diffusion model can supervise a LoRA from paired text conditions, without a large image collection, by aligning denoised scores under a target concept with scores under an attribute-augmented concept. The present method belongs to the same family: the pretrained model is both source of knowledge and teacher. It differs in three respects. It targets a flow-matching DiT rather than an epsilon-prediction UNet. It refuses to back-propagate through the image backbone. And it treats style as a shift to be absorbed in the text-injection maps, which is a weaker but computationally local surrogate of score-space alignment.

2.3 Krea 2 conditioning path

Krea 2 is a dense 12B single-stream DiT. Language is encoded by Qwen3-VL-4B with multi-layer feature aggregation. Those features are projected by `txt_in` (two linear maps; native names `txtmlp.1` and `txtmlp.3`) and mixed by a small TextFusion transformer whose public linear is `text_fusion.projector`. Community loaders that split reference LoRAs by layer group treat text-fusion and txt_in as the site of reference addressing and style transfer. That empirical split is the architectural reason a text-only extraction can hope to produce a visible style prior at all.

2.4 Why the original velocity pipeline fails on CPU

Let N_c be the number of semantic concepts, N_z the number of seeds, N_t the number of timesteps, and two prompts per pair. The original proof-of-concept budget of 10–20 concepts, 2–5 seeds and 4–8 timesteps already implies on the order of 10^2 to 10^3 DiT forwards before any distillation begins. Each forward materializes a velocity tensor at image resolution in latent space. Distillation then repeats a student forward and a teacher forward at every optimizer step. On a CPU this is not slow research; it is a stalled pipeline. The redesign therefore treats DiT execution as an external service, not as a local primitive.

3. Problem Statement

Let P_c be a content prompt and P_{c+s} the same content with a style phrase inserted. Let $E(\cdot)$ denote the official Krea 2 text encoder, including the multi-layer aggregation used at inference. Let $Proj_W$ denote the composition of the text-injection linear maps with weights W drawn from RAW. The original study asked for a low-rank DiT update satisfying a behavioral identity in velocity space. Two issues must be repaired before that identity can even be discussed.

First, the draft objective that set $F_{\theta+\Delta\theta}(x_t, t, P) \approx D_s$ is dimensionally and semantically inconsistent. D_s is a difference of velocities, not a velocity. The internally consistent teacher–student target, if one were willing to run the DiT, would be

$$F_{\theta+BA}(x_t, t, P_c) \approx F_\theta(x_t, t, P_{c+s})$$

The adapter on a content-only prompt should reproduce the base model on the styled prompt.

Second, even the repaired objective is inaccessible without a GPU. The feasible surrogate is therefore restricted to the maps that consume E(P):

$$Proj_{W+BA}(\, E(P_c)\,) \approx Proj_W(\, E(P_{c+s})\,)$$

Hold the encoder and the DiT fixed. Edit only the text-injection linears.

A solution of this surrogate cannot be expected to reconstruct brush-level style. It can be expected, if RQ1 holds, to push the model's default conditioning toward the same region of text-feature space that the style phrase already occupies.

4. Method

4.1 Overview

The pipeline has five stages: prompt-pair construction, text-only encoding, style-direction estimation, closed-form LoRA synthesis, and external image validation. No stage runs the DiT or the VAE on the local machine.

Prompt pairing → Text-feature difference → Style subspace

→ Closed-form ΔW = BA → Hosted image validation

4.2 Prompt-pair construction

A target style is admitted only if the base model already responds to it when the style phrase is written explicitly. Watercolor, oil painting, pencil sketch, and generic anime rendering are admissible starting points. A private studio look that the model cannot produce from text is out of scope; there is then no internal representation to extract.

Each experiment uses a bank of 20 to 40 content prompts spanning objects, scenes, and figures, held out from a disjoint evaluation bank. Every content prompt is paired with one or more style formulations to reduce token idiosyncrasy:

```
watercolor / watercolor painting / traditional watercolor illustration / hand-painted
watercolor
```

Insertion is prefix-style, not a free paraphrase of the whole sentence, so that the residual difference is dominated by the style span rather than by a rewritten scene.

4.3 Text-only encoding

Pairs are encoded with the official Qwen3-VL-4B checkpoint used by Krea 2, in text-only mode, on CPU. The vision tower is not loaded. Features are taken from the same intermediate-layer aggregation that Krea 2 uses at inference. Substituting CLIP or a different Qwen checkpoint produces a direction that cannot be legally added to Krea's `txt_in`.

Style phrases change sequence length. A naive subtraction of two variable-length feature matrices mixes a genuine style shift with a token-alignment artifact. The encoder output is therefore split. Shared content tokens are aligned by longest common subsequence on the tokenized strings. The residual style span is pooled separately. Two statistics are stored for each pair i:

$$\Delta_i^{pool} = \mathrm{pool}(E(P_{c_i+s})) - \mathrm{pool}(E(P_{c_i}))$$

$$\Delta_i^{span} = \mathrm{pool}(E(P_{c_i+s})[\text{style tokens}])$$

4.4 Testing for a shared style direction

A single pair cannot establish that the shift is stylistic. Pairwise cosine similarities $\cos(\Delta_i^{pool}, \Delta_j^{pool})$ are computed across contents. PCA is then run on the matrix whose rows are the pooled differences. Two gates decide whether to synthesize a LoRA.

Gate	Statistic	Proceed if
Directional consistency	Mean pairwise cosine of pooled differences	Mean cosine is clearly above a near-zero noise floor; a working target is 0.4
Rank concentration	$\lambda_1 / \Sigma_k \lambda_k$ from PCA	The leading component explains a disproportionate share of variance

Table 1. Cheap numerical gates that abort a style before any LoRA file is written.

If both gates fail, the style phrase is not a stable concept in the encoder and closed-form absorption will fit residual semantics. The experiment stops. This is a feature of the method, not a missing ablation: CPU time should be spent only on styles the model already owns.

4.5 Closed-form absorption into text-injection maps

Let W $R^{d\_out \times d\_in}$ be one text-injection matrix, read from the RAW checkpoint by named-key lazy load. The matrices of interest are `txt_in.linear_1`, `txt_in.linear_2`, and `text_fusion.projector`. The 28 DiT blocks are never opened.

Write h for a pooled or token-wise content feature and μ for the estimated style shift (the mean of Δ_i^{pool}, or a leading principal component). The desired first-order behavior is that the styled projection applied to content features reproduce the base projection applied to styled features:

$$(W + \Delta W)\, h \;\approx\; W\,(h + \alpha\mu) \;=\; W\,h + \alpha\, W\mu$$

which is satisfied by any ΔW whose action on typical content features equals α Wμ.

Let $\bar{h}$ be the mean pooled content feature over the extraction set, and set $v = \bar{h} / \|\bar{h}\|^2$ so that $v^T h \approx 1$ on typical prompts. The rank-1 solution is then

$$\Delta W = \alpha\,(W\mu)\,v^T, \quad B = \alpha\, W\mu, \quad A = v^T.$$

When PCA retains r leading components $\mu_1, \ldots, \mu_r$, the same construction is applied to each component and the factors are concatenated, producing a rank-r adapter with r $\{1, 2, 4, 8\}$. Rank 1 is the default proof of concept. Rank 8 is the escalation if visual inspection shows only a color shift and no change in rendering tendency.

4.6 Least-squares refinement without a DiT

The rank-1 formula uses a single mean feature as a surrogate for the identity constraint $v^T h = 1$. A tighter fit, still on CPU, replaces that surrogate with an explicit thin factorization on cached features. Let $H_c^{(i)}$ and $H_{c+s}^{(i)}$ be aligned feature matrices for pair i. For each target map W one solves

$$\min_{B,A} \;\; \Sigma_i \,\|\,(W + BA)\, H_c^{(i)} - W\, H_{c+s}^{(i)}\,\|_F^2.$$

The problem is an ordinary low-rank approximation of the implied residual map from content features to the desired output increment. It is solved by SVD of the stacked residual, or by a few alternating least-squares sweeps. Gradients never enter the encoder and never enter the DiT. Sequence-length mismatch is handled by the same content-token alignment used in Section 4.3; unaligned style tokens contribute only through their pooled increment.

4.7 Packaging a loadable LoRA

Krea 2 LoRAs circulate under two key namespaces. Native and Musubi exports use names such as `txtmlp.1` and `txtfusion.projector`. Diffusers and PEFT exports use `txt_in.linear_1` and `text_fusion.projector`. The synthesizer writes one namespace and, if needed, a second file with the aliased keys. Scaling follows the common convention lora_alpha = rank, so that a hosted slider of 1.0 corresponds to the constructed ΔW. Tensors are stored in bf16 or fp16. No dummy keys are written for untouched DiT blocks; loaders that ignore missing targets will apply a text-only adapter cleanly.

5. Experimental Protocol

5.1 Local compute budget

The intended local machine has a CPU, enough RAM to hold a quantized 4B text encoder (on the order of 4–8 GB for a 4-bit or 8-bit build), and disk access to the RAW safetensors file. The DiT weights other than the named text-injection matrices are not materialized. Image decoding is never performed locally.

5.2 Phase plan

Phase	Goal	Local work	Stop rule
0. Admissibility	Confirm the base model already responds to the style word	None. Hosted generations of P_c versus P_{c+s}	If explicit prompting does not change style, abandon the target
1. Direction discovery	Estimate a stable text-feature shift	Encode 20–40 pairs; cosine and PCA	Fail the gates in Table 1
2. Closed-form LoRA	Write rank-1 and rank-4 adapters on txt_in + projector	Lazy-load three matrices; SVD or rank-1 formula	None; files are cheap
3. External validation	Test whether the adapter creates a default style prior	Upload .safetensors to Tensor.Art, Krea, or Comfy-on-a-host	See Section 5.3
4. Escalation	Recover rendering, not only palette	Raise rank to 8; add fusion block linears	If still palette-only, the surrogate has hit its ceiling

Table 2. A four-phase plan that spends CPU time only after a style has cleared cheap gates.

5.3 Four-condition visual test

All images are generated on a hosted Krea 2 Turbo endpoint. Seeds and samplers are frozen across conditions. The evaluation prompt is drawn from the held-out bank and does not appear in extraction.

Condition	Prompt	Adapter	What it isolates
A. Baseline	a girl walking through a forest	off	The model's default prior
B. Explicit style	a watercolor girl walking through a forest	off	The native style concept
C. Proposed LoRA	a girl walking through a forest	on	Default-style transfer without the style word
D. Combined	a watercolor girl walking through a forest	on	Interference or stacking with the native concept

Table 3. The original four-condition protocol, retained because it separates the right questions.

The decisive comparison is A → C. If C moves toward B in palette, edge character, and surface quality, the adapter has converted a conditional behavior into a prior. B → C asks how much of the native concept was recovered; identity is not required. D detects destructive interference. Strength is swept through {0.25, 0.50, 0.75, 1.00, 1.25}. A monotone response to the sweep is supporting evidence that the update is a direction rather than noise.

5.4 Generalization split

Extraction concepts and evaluation concepts are disjoint. A compact recommended split is:

Role	Concepts
Extraction	tiger, castle, woman, spaceship, fruit, motorcycle, jungle path, harbor
Evaluation	dog, portrait, train, landscape, robot, food, library interior, night market

Table 4. Content split used to test whether the adapter is a style prior or a concept-specific hack.

5.5 Quantitative companions to visual inspection

Hosted platforms rarely expose velocity tensors, so image-space proxies are used. A CLIP or SigLIP direction between A and B defines an empirical style axis; C is projected onto that axis. The same encoder can report content preservation between A and C. Neither metric replaces looking at the pictures. They exist to make a negative result harder to argue away and a weak positive result harder to overclaim.

5.6 Baselines

Five baselines keep the claim honest. (1) No adapter. (2) Explicit style prompt, no adapter. (3) Official image-trained style LoRAs released for Krea 2, such as the watercolor adapter in the public collection, when the target style has an official counterpart. (4) A random rank-matched LoRA on the same target modules, to reject the hypothesis that any text-side perturbation looks like style. (5) A mean-offset embedding injected at encode time without being packaged as LoRA, which tests whether the closed-form packaging step itself loses information.

6. What Success Can and Cannot Mean

A successful C condition is not an aesthetically maximal watercolor. It is a measurable drift of the default prior toward the native style concept, on held-out contents, with a monotone strength sweep, obtained without images and without a GPU. That is a weaker object than a production style LoRA, and it is the object this method is allowed to claim.

Expected positive behavior includes a shift in palette, paper or pigment atmosphere, edge softness, and global rendering tendency. Expected failure modes are equally specific. The adapter may tint without changing mark-making. It may bind the style to objects that co-occurred with the style phrase in pretraining. It may distort structure if the pooled shift still contains residual semantics. It may transfer poorly from RAW-derived weights to Turbo if Turbo's text path was altered by distillation, which is why validation must be run on Turbo rather than inferred from RAW algebra.

7. Limitations

Unknown styles. If the base model has no usable representation of the target, the encoder difference is not a style direction. The method reorganizes existing knowledge.

Surrogate gap. Aligning text-injection maps is not the same as aligning velocity fields. High-frequency style that lives in mid-to-late DiT blocks cannot be written by editing `txt_in`. This is the price of refusing to run the backbone.

Linearity. The construction assumes that a style phrase acts as an approximately additive shift in feature space. Styles that rewrite composition, camera, or object identity as a function of content will leak into μ and then into the LoRA.

Key and aggregation fidelity. The method is only as correct as the encoder aggregation and the weight-key mapping. An unofficial text encoder, a missed layer in the multi-layer stack, or a swapped linear_1 / linear_2 produces a file that loads and does nothing useful.

RAW-to-Turbo transfer. Official image LoRAs are trained so that RAW updates apply on Turbo. A closed-form text-side update is not covered by that training contract and must be tested.

Hosted evaluation bias. External platforms may apply unpublished post-processing. The four-condition protocol should be run inside one platform with fixed settings, not compared across platforms.

8. Discussion

The original study changed the role of the pretrained model from student to teacher. That inversion is worth keeping. What does not travel is the assumption that the teacher signal must be a velocity. On a CPU, the only teacher signal one can actually collect is the signal that lives in the language pathway. Packaging that signal as LoRA is a way of turning a conditional token into a prior: the user no longer has to write *watercolor* because the adapter permanently biases the maps that would have read that word.

This is a form of representation engineering, not a form of dataset-free creative invention. It is closer to absorbing a soft prompt into weights than to training a style model. The distinction matters for evaluation. Comparing the method against an image-trained official LoRA and declaring defeat because the brushwork is weaker would miss the claim. Comparing it against the explicit style prompt and asking how much of that prompt can be retired is the correct test.

If later access to a GPU becomes available, the repaired velocity objective in Section 3 can be restored as a second stage, initialized from the text-side LoRA produced here. The CPU method is then not a competitor to Concept Sliders; it is a feasible initializer and, for many users, the only complete method they can run.

9. Broader Use

Any named visual attribute that the official encoder represents stably is a candidate: cinematic photography, oil painting, pencil sketch, vintage photography, concept-art lighting, editorial illustration. Attributes that the model treats as objects rather than renderers — a specific living artist, a private product finish — are not. The same pipeline can in principle emit a negative adapter by flipping the sign of μ, which would suppress a style the model otherwise applies too freely. That experiment is optional and should not be conflated with the main feasibility claim.

10. Conclusion

Extracting a style LoRA from a pretrained text-to-image model without images is a reasonable research goal. Doing it by differencing the velocity field of a 12B flow-matching DiT is not a CPU method. This paper relocates the same goal to the text-conditioning pathway of Krea 2. Paired prompts are encoded with the official language model. A cross-content shift is estimated and gated by cosine and PCA. A closed-form low-rank update is written into `txt_in` and `text_fusion`. The file is validated on a hosted Turbo endpoint with a four-condition protocol that asks whether the style word can be removed from the prompt.

The primary feasibility question is therefore no longer whether a pleasing picture can be made. It is whether a style that already lives in the model's language pathway can be isolated well enough to become a reusable, content-independent, low-rank parameter modification, on a machine that cannot run the image backbone at all. If that narrower demonstration succeeds, it is evidence that some forms of style adaptation are reorganization of knowledge the model already has — and that the cheapest place to reorganize that knowledge is the place where language first becomes conditioning.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” ICLR, 2022.

[2] R. Gandikota, J. Materzynska, T. Zhou, A. Torralba, and D. Bau, “Concept Sliders: LoRA adaptors for precise control in diffusion models,” 2023.

[3] Krea AI, “Krea 2,” model card and technical documentation for RAW and Turbo checkpoints, 2026. https://github.com/krea-ai/krea-2

[4] Krea AI, “Krea 2 RAW,” Hugging Face model card, 2026. https://huggingface.co/krea/Krea-2-Raw

[5] Hugging Face Diffusers, “DreamBooth LoRA training for Krea 2,” README_krea2.md, 2026.

[6] kohya-ss, “Musubi Tuner: Krea 2 LoRA training notes,” including default targets of 264 Linear layers, 2026.

[7] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” ICLR, 2023.

[8] Qwen Team, “Qwen3-VL,” technical report and 4B instruct checkpoint used as Krea 2 text encoder.

[9] Comfy-Org, “Krea 2 pack and official style LoRA collection,” Hugging Face, 2026.

[10] A. Radford et al., “Learning transferable visual models from natural language supervision,” ICML, 2021.

[11] Original feasibility draft, “Zero-shot style extraction and LoRA distillation from a pretrained text-to-image model: a feasibility study on extracting model-internal style representations from Krea 2 RAW.”

Appendix A. Target modules and key aliases

The synthesizer should prefer the smallest module set that can absorb a style prior. The recommended order of inclusion is projector, then the two txt_in linears, then fusion block linears only if Phase 4 is triggered.

Diffusers / PEFT key	Native / Musubi key	Role
txt_in.linear_1	txtmlp.1	First text-width projection
txt_in.linear_2	txtmlp.3	Second text-width projection
text_fusion.projector	txtfusion.projector	Layer-mixer into the DiT stream
text_fusion.layerwise_blocks.*	txtfusion.layerwise_blocks.*	Optional escalation
text_fusion.refiner_blocks.*	txtfusion.refiner_blocks.*	Optional escalation
transformer_blocks.*.attn.to_q / to_k / to_v / to_out.0	blocks.*.attn.wq / wk / wv / wo	Do not touch on CPU

Table 5. Key aliases. Only the first three rows are used in the default CPU method.

Appendix B. Worked rank-1 construction

Given cached features, the default adapter is obtained as follows.

1. Encode each pair with the official text-only stack. Align content tokens. Store pooled content features h_i and pooled differences Δ_i.
2. Set $\mu = (1/N)\ \Sigma_i\ \Delta_i$ and $\bar{h} = (1/N)\ \Sigma_i\ h_i$. Normalize the gate: if mean pairwise $\cos(\Delta_i, \Delta_j)$ is near zero, stop.
3. For each target matrix W, compute $u = W\mu$ and $v = \bar{h} / \|\bar{h}\|^2$.
4. Write $B = \alpha\ u$ as a column and $A = v^T$ as a row. Default $\alpha = 1$.
5. Serialize A and B under the chosen namespace with lora_alpha = 1 for rank-1, or lora_alpha = r after concatenating r components.
6. Validate on Turbo with Table 3. Sweep strength. Do not iterate on extraction prompts that appear in the test set.

Appendix C. Relation to the original draft

Original draft	This paper
Teacher signal is a velocity difference D_s	Teacher signal is a text-feature difference Δ
Objective accidentally targeted D_s as a model output	Corrected objective: match styled conditioning from content input
PCA on velocity tensors across seeds and timesteps	PCA on encoder differences across contents only
LoRA trained by backprop through the 12B DiT	LoRA written by rank-1 formula or thin SVD
Target module search over attention, MLP, and fusion	Default targets restricted to txt_in and text_fusion
CPU pipeline by skipping VAE decode	CPU pipeline by never loading the DiT blocks
Claim framed as general zero-shot style creation	Claim framed as reorganization of an already-known style prior

Table 6. Explicit mapping from the motivating draft to the CPU-feasible redesign.

The original research question is unchanged. The feasible method is not. If a later implementation shows that C consistently approaches B on held-out prompts, the narrower demonstration is enough: some visual styles in Krea 2 are already linear enough, in language space, to be packaged as a LoRA without ever generating a training image.